[Vision Paper] Towards AI-Conducted Benchmarking: Autonomous Performance Experiments of Data Systems in Kubernetes

Patrick K. Erdelt
Berliner Hochschule fuer Technik (BHT)
Berlin, Germany
patrick.erdelt@bht-berlin.de

Abstract

Large language model agents already write SQL, tune configuration knobs, and diagnose incidents in cloud environments — yet performance benchmarking, the empirical backbone of systems research, remains designed exclusively for human experimenters. We argue that benchmarking should become AI-ready, and envision the agent as the experimenter: autonomously conducting, interpreting, and iterating on reproducible benchmark studies. The key enabler is a two-sided contract between agent and benchmarking infrastructure: on the input side, a catalog of workloads *and* of systems under test that exposes intent, parameters, and the configuration choices a fair comparison depends on; on the output side, a self-describing result folder bundling metrics, provenance, and validity indicators. This inverts the prevailing paradigm of agentic environments, in which infrastructure evaluates agents. We describe a prototype realization on a Kubernetes-based benchmarking framework, demonstrate the feasibility of a closed experiment loop for a DBMS comparison study, and outline a research agenda spanning contract standardization, guardrails against invalid experiments, and what reproducibility requires once experiments are designed by a model.

Keywords

Benchmarking, Kubernetes, DBMS, LLM Agents, Reproducibility, Experiment Orchestration

1 Introduction

> *Our nightly ETL runs 30 % slower since the platform upgrade — which layer is responsible, and why?*

Every practitioner has asked a question of this shape, and no tool answers it end to end, because answering requires conducting a benchmark *study*: selecting instruments, designing discriminating experiments, judging validity, iterating. Orchestration frameworks have automated the *execution* of such studies [10], but their workloads are parameterized from documentation and folklore, and their results interpreted from logs whose meaning lives in expert heads. Meanwhile LLM agents write SQL, tune knobs [21, 22], and diagnose incidents in Kubernetes clusters [5] — but cannot yet *conduct an experiment*. This is not a capability gap of the agents but an interface gap of the infrastructure: benchmarking is not AI-ready.

This paper envisions the agent as the experimenter, enabled by a deliberately small, two-sided contract: a catalog of workloads and systems together with a generated environment descriptor lets the agent express an experiment as a validated specification, and a result contract turns the experiment summary into a navigable entry point – provenance, validity checks, links into the result folder – for a bounded-context reader.

The consequence is a change in what benchmarking *is*: from an expensive, occasional, human-triggered campaign to a standing capability in which experiments answer events. A new DBMS release lands overnight — by morning, an agent has compared it against the archive and designed the experiments that isolate a regression's cause. A puzzling slowdown becomes a topology study, with placement, storage, and co-location as verified factors. A production anomaly is replayed as a controlled counterfactual; new hardware triggers its own characterization; and a published study, archived as specification and contract, is re-run by a reviewer's agent on another cluster. The first make benchmarking continuous within an organization; the last turns reproducibility from a badge into an executable check. And they are deliberately the *near* end of a longer road: the opening question — open-ended, cross-layer, unconstrained by any catalog — marks the horizon, and further still lie questions that turn benchmarking into decision-making under a budget: *what is the cheapest configuration that keeps p99 latency under 200 ms through next year's growth?* Our research agenda charts what is still missing between here and there.

We contribute the vision and its use cases, the two-sided contract as its enabling abstraction, a prototype realization on an existing orchestration framework [12] with a feasibility demonstration of the closed loop, and a research agenda for the trust such an experimenter must earn and the reach it gains.

2 Related Work

Four research lines converge on our vision; for each, we state its ambition, its state of the art, and what it leaves open — the gap paragraph at the end of this section collects why their combination is both missing and necessary.

2.1 Reproducible Benchmarking of DBMS in the Cloud

The vision of this field: any performance claim can be re-established by anyone, anywhere — the experiment as a portable, repeatable artifact rather than expert craft. The TPC family standardizes *what* to measure; OLTP-Bench/BenchBase [8], Bexhoma [9, 10, 12], and related frameworks [15, 31] automate *how*, with cloud-native adoptions turning benchmarking into a scalable, automatable process [11], with SPEC principles adding methodological rigor [27]. The convergence is broad — Kubernetes operators [18], declarative experiment specifications on Kubernetes [6, 19], and commercial benchmarking-as-a-service [30] — but the contrast is exact: each runs the experiment it is *given*, whereas our contract exists so an agent can decide what to give it. Execution is thus

EDBT '27, Lille (France)

repeatable and portable — the experimenter is not: workload selection, parameterization, and interpretation remain expert work that demands deep benchmark knowledge [4], and in which even experts fall into documented pitfalls [13, 28] — evidence that validity checking must be systematic, not intuitive. Our contract targets exactly this remaining human-only interface.

2.2 LLM Agents for Database Tuning

This line pursues the self-managing database, ending expert administration as the price of good performance. Beyond learned tuners [1], LLMs mine manuals for tuning hints [32], refine Bayesian search spaces [21], tune analytical workloads via prompts [14], map workloads directly to configurations [16], emulate a DBA with cooperating agents [22], mine tuning knowledge from DBMS source code [37], and diagnose anomalies [38]. Most radically, code synthesis now generates entire workload-specialized OLAP engines and cardinality estimators, guided by iterative performance evaluation [33, 34], and multi-agent tuners even arrive independently at machinery close to ours — a structured performance digest, fail-closed checks [24]. All share one objective and hence one interface shape: scalar optimization of a single system, through an *implicit* input contract (mined manuals, learned features, prompts) and a result channel reduced to a reward — no provenance, placement, repetitions, or validity, because the benchmark is trusted as an oracle; an invalid run is noise to the optimizer, not a finding. This cannot express an experiment — no factorial design, no cross-system comparison, no trust judgment — so each paper rebuilds a bespoke harness; database gyms [23] standardize harnesses, but as training-data factories for self-driving components. We invert the benchmark's role — from disposable oracle to object of design and interpretation — and replace the implicit scalar interface with an explicit, versioned contract these systems could adopt as substrate — the more systems become tunable, and now synthesizable, in hours, the more evaluation becomes the bottleneck.

2.3 Agentic Environments for Cloud Operations

The ambition here is the autonomous cloud: infrastructure that detects, diagnoses, and repairs its own incidents. AIOpsLab deploys applications, injects faults, and exposes a standardized *agent–cloud interface* [5]; ITBench replicates realistic incidents [17]. These are the closest architectural relatives of our contract — a curated boundary between probabilistic agent and cluster — but with inverted purpose: the environment is instrumented to *evaluate the agent*, whereas we instrument benchmarking so the agent can *evaluate a system under test*. The inversion changes what the boundary must express: experimental-design vocabulary and validity metadata instead of incident context and remediation actions. Tellingly, AIOpsLab observes that agents struggle less with data availability than with interpreting the telemetry they are given [5] — precisely what our result-side contract addresses.

2.4 Autonomous Experimentation and Machine-Actionable Results

The broadest vision is science at machine speed: discovery limited by ideas, not by the labor of experimentation. Agent frameworks automate the hypothesis–experiment–analysis loop [25, 29]. Data-science agents conduct its *observational* form over existing data, building data-to-insight pipelines over data lakes — where benchmarks such as KramaBench show they still fail at end-to-end orchestration [20]. Our agent is their experimental counterpart: it does not analyze data that exists, but designs interventions that create data, under physical resource constraints where an invalid run costs cluster-hours — while the interpret phase of our loop is itself a data-agent task over the result folder, making data agents a component of this vision rather than a competitor. From the physical side, meanwhile, the Lab Agent Protocol standardizes the agent-to-instrument edge with capability descriptions and typed, uncertainty-bearing results [39] — structurally our environment descriptor and result contract, for instruments instead of clusters. Machine-actionable metadata has matured for data — FAIR [35], Croissant [2], an MLCommons benchmark ontology [26] — but none covers systems performance experiments, whose factorial configurations, per-run metrics, placement provenance, and validity checks have no comparable representation today. Closest are systems that *extract* benchmarks from production workloads [7] or audit whether benchmark artifacts are correct [36] — treating the benchmark as an object to build or verify, as we do, but never handing the result to an autonomous experimenter.

Research gap and contribution. The volume of adjacent work is not evidence this vision is taken but that the field keeps reaching for it blindly: every system above had to run a benchmark and read its result, and each built a private, throwaway way to do so [6, 24] — instancing the same interface without naming it. No one has treated the boundary between agent and benchmarking infrastructure as an object worth building once. That recognition is our contribution: a two-sided contract — catalog of workloads and systems in, self-describing result folder out — that turns the throwaway harness into a standing one and the agent from a tuner into an experimenter. The dense prior work clusters entirely on the optimization side of that line; past it we find open ground.

3 The Two-Sided Contract

By a *contract* we mean a versioned, machine-readable agreement between agent and benchmarking infrastructure that fixes what each side may express and what it may assume. Three properties separate it from documentation. It is *enforced*: validation rejects what the contract does not cover, and generation guarantees that what it promises is present — which is also what turns the with-/without comparison into a measurable variable rather than a preference. It is *versioned* and archived with every experiment, so agent behaviour is auditable against the contract it actually saw, unlike knowledge held in model weights, which is unversioned, unauditable, and cluster-blind. And it is *inspectable* by humans and machines alike. The term is converging on us from elsewhere: engine synthesis binds a workload to a generated system through what its authors likewise call a contract [33], and agent tooling there is similarly defined as a closed set of tools that forms the entire agent–system boundary. Their contract binds a workload to a synthesized system; ours binds an experimenter to a benchmarking infrastructure. The hard part of such a contract is not its syntax but its content. What an instrument measures and what it cannot, which parameter combinations are meaningful, what invalidates a run, which result is implausible — this is decades of accumulated community knowledge, the reason benchmark creation has been called an art [3]: our field has long written down its lessons for human readers [4, 13, 27, 28]. Restating that corpus in a form machines can act on is a community-scale program, comparable to what the standardization of workload definitions

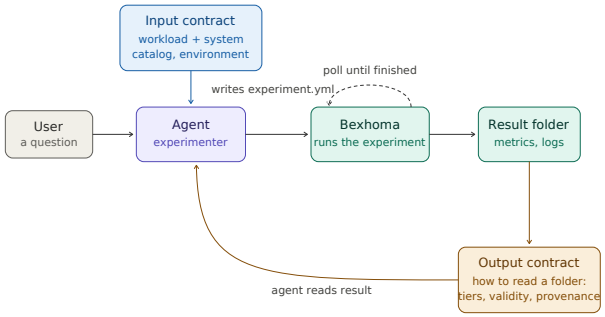

Figure 1: The loop the contract enables. A question reaches the agent, which uses the input contract — the workload and system catalog and the generated environment descriptor — to write a validated `experiment.yml`; Bexhoma runs it, polled until finished; the result folder is read back through the output contract. The two contracts frame the two legs of the loop — expressing an experiment and interpreting its result.

once was. This paper contributes the frame and a seeded catalog, not the finished corpus.

Figure 1 shows the loop the contract enables; Figures 2–5 then illustrate each artifact end to end, and the following subsections state each artifact and its objective.

3.1 Workload and System Catalog

The catalog spans two independent axes: *what* to run and *what to run it on*. The workload contract is the instrument — its objective is to let the agent map a goal-level question to the right one, stating *intent* before parameters: why a workload exists, when to choose it over its alternatives, its explicit non-scope, and its cost (Figure 2). The system contract is the system under test, and it spans a spectrum of configurability: at one end a managed database service exposes nothing but an endpoint; at the other, a study demands a clean deployment at a pinned version, a configuration file, chosen knob values, physical design (indexes, statistics, storage format), resource limits, and hardware placement. This spectrum is where the confounders of a fair comparison live — defaults versus tuning, comparable memory accounting, indexed versus not — each of which the contract turns into a recorded field rather than a tacit decision. An experiment references one entry of each kind: the same workload runs against many systems, and the same system serves many workloads.

3.2 Environment Descriptor

Objective: ground experiment designs in the actual cluster rather than in assumptions inherited from training data. The descriptor is *generated* from the cluster at experiment time and curated for benchmarking relevance.

3.3 Experiment Specification

Objective: make the agent's experimental design an explicit, validated, archivable artifact. The specification is the interchange format of the loop: written by the agent from catalog and descriptor, checked by validation, and stored with the results as provenance (Figure 4). It records not only *what* to run but *why*: a hypothesis field states what the experiment is meant to discriminate.

```yaml
# catalog.yml (excerpt) -- contract v0.1
# two axes: WHAT to run, WHAT to run it on
workloads:
  tpch:
    why:  analytical join & aggregation,
          22 queries
    when: join-heavy or reporting tasks;
          NOT point reads or updates
    cost: load ~10 min per sf and system
    supports: [postgresql, pg_duckdb]
    parameters:
      queries:
        type: subset of [1..22]
        semantics: 5,7,8,9,21 = multi-way
          joins; 1,6 = scan-dominated
      mode:
        type: enum [power, throughput]
        semantics: power = one query stream;
          throughput = concurrent streams
      streams:
        type: int | list   # list = factor
        condition: {mode: throughput}
  ycsb:
    why:  key-value CRUD, tunable r/w mix
    when: raw KV throughput; NOT joins
    # ... omitted ...
systems:
  managed-pg:            # one end: nothing
    endpoint: my-svc:5432   #  to configure
  postgresql:            # the other end
    image: postgres:16.3
    knobs:
      shared_buffers:
        type: size
        why: buffer cache; largest memory
          knob
      random_page_cost:
        type: float
        default: 4.0
        why: planner cost of random vs.
          sequential read; lower on SSD
      # ... omitted ...
    profiles:
      analytical-ssd:
        why: OLAP on node-local NVMe,
          sized from the memory limit
        requires: {storage_class: ssd}
        derive:
          shared_buffers: 0.25*memory_limit
          effective_cache_size:
            0.75*memory_limit
          random_page_cost: 1.1
```

Figure 2: Input contract (excerpt): one catalog, two axes. Workload entries state *why*/*when*/non-scope down to query-level intent and declare which systems they support; system entries span a spectrum from a managed endpoint to a fully tunable deployment. Profiles derive knob values from the experiment's resource limits and declare their preconditions — so the confounders of a fair comparison, tuning parity above all, become explicit and checkable rather than tacit.

3.4 Validation

Objective: functional enforcement of the input contract. A dry-run mode checks a specification against workload semantics and the environment descriptor before any cluster resource is touched, and rejects with structured, machine-readable errors the agent can self-correct from.

```yaml
# environment.yml -- generated from the
# cluster at experiment time
cluster: bht-a328
node_groups:
  benchmark:
    nodes: 4
    cpu: 32
    memory: 64Gi       # hard ceiling
    semantics: reserved for SUTs, no
      co-scheduling by default
  driver:
    nodes: 2
    cpu: 16
    memory: 32Gi
storage_classes:
  local-ssd:
    semantics: node-local NVMe, fast,
      not portable across nodes
  ceph-rbd:
    semantics: network-attached, expect
      latency sensitivity
# no system definitions here: the
# environment describes cluster
# resources, the catalog the systems
```

Figure 3: Environment descriptor (excerpt): generated from the cluster, curated for benchmarking relevance — entries carry experiment-relevant semantics, not only capacity.

```yaml
# experiment.yml -- written by the agent,
# validated before touching the cluster
title: join performance under concurrency
hypothesis: pg_duckdb's edge on joins narrows
  under concurrency; rival: it is memory
  starvation per stream, not thread
  contention -- raising the limit would
  then recover it
discriminates: [streams, sut.memory_limit]
workload: tpch
queries: [5, 7, 8, 9, 21]  # multi-way joins
mode: throughput
streams: [1, 4]            # factor
scaling_factor: 10         # 10 GB database
repetitions: 3
sut:
  system: [postgresql, pg_duckdb]  # factor,
  profile: analytical-ssd    # catalog entry
  memory_limit: [32Gi, 64Gi]       # factor
  node_group: benchmark
# both systems run the same named profile:
# tuning parity is stated, not assumed
```

Figure 4: The agent's experiment specification: declarative, diffable across loop iterations, archived with the results — and stating, before the run, the hypothesis and the rival explanation its factors are chosen to separate.

3.5 Result Contract

Objective: let a bounded-context reader move from aggregate to evidence without human guidance. The generated summary is the declared entry point; tiered file groups, a naming legend, and links into the result folder define the navigation; validity tests precede any metric (Figure 5). Some of these tests are absolute (a restarted container invalidates a run); others are comparative, deriving plausibility corridors from the archive of past results for the same workload and hardware. The comparative ones have a boundary worth naming: a system the archive has never seen — often the very reason for the study — has no corridor, and the contract must say so rather than silently fall back to the nearest neighbour.

```
## Connection PGDuckDB-1-2-1
node: cl-node-07  (requested: benchmark)
details: query_times_PGDuckDB-1-2.csv
monitoring: metrics/mem_PGDuckDB-1-2.csv

## Tests
[ok]   planned == actual workflow
[ok]   no SUT container restarts
[warn] Q21 3.4x the PostgreSQL baseline
       within this experiment (312 s)
       -> logs/driver-PGDuckDB-1-2-1.log
[skip] archive corridor: no history for
       pgduckdb on this hardware class

## Provenance
spec: experiment.yml  (submitted copy)
images: postgres:16.3, pgduckdb:17-main
```

Figure 5: Output contract (excerpt of a generated summary): every section links into the result folder, and validity tests precede any metric. A test that cannot be applied — here the archive corridor, for a system without history — is reported as skipped rather than omitted.

3.6 From Numbers to Explanations

Objective: support what distinguishes an experimenter from a measurement device — explaining effects, not only obtaining numbers. The contract carries explanation at three layers: within-experiment attribution over joinable metrics and monitoring data, hypothesis-discriminating follow-up experiments in the loop, and the hypothesis field that turns a trajectory into a documented investigation.

4 Prototype and Demonstration

We realize the contract on top of Bexhoma, a Kubernetes-native orchestrator for benchmark experiments on data systems [10, 11]. It deploys systems under test, loaders, drivers, and monitoring as cluster workloads, and its deployment configuration is expressive enough to treat placement, storage class, resource limits, and system configuration as experimental factors rather than fixed settings. Being cloud-native, it records what it does: metrics per component and phase, logs of every deployment step, planned versus actual workflow, and the full configuration of each run are collected into one result folder per experiment. That instinct — log everything, keep it together, keep it reproducible — is what makes the output side of the contract a matter of *describing* what is already there rather than instrumenting what is missing.

The prototype's running example is a single user task:

> *Is pg_duckdb better on joins than PostgreSQL, even under concurrency? I have a 10 GB dataset, and our servers have at most 64 GB of RAM.*

A deliberately simpler question than the one we opened with — one instrument family, two systems, one cluster — but not a textbook comparison: vectorized execution that wins a single query stream is also the design most exposed when streams contend for threads and the memory budget per stream shrinks, so the outcome is open. It exercises every artifact in sequence: the catalog's intent fields rule out YCSB (non-scope: joins) and select TPC-H with a join-heavy subset, in throughput mode with concurrent streams (Figure 2); 10 GB maps to scale factor 10 and the memory budget to limits validated against the environment descriptor; the specification declares system, stream count, and memory limit as factors; and the result contract carries per-query

attribution over joined monitoring data. The two factors also discriminate between rival explanations within a single experiment: if concurrent degradation is memory starvation, raising the limit recovers it; if it is thread contention, it does not. Where evidence still underdetermines the cause, the loop continues with a follow-up — a storage-class variation, or a hardware microbenchmark against the same volume. Throughout, the prototype plays the role proper to a vision paper's evidence: it is not the vision realized, but the argument that the vision is testable — and near.

5 Research Agenda

Cheap experiments are arriving whether or not we specify them well; what this agenda decides is whether the result is trustworthy or merely abundant. The danger is not new — selective reporting, unreplicable margins, and results too favourable to their author long predate language models, which is why this community built audited benchmarks and wrote down its pitfalls [13, 28] — but the speed is, and the governance we inherited assumes a human bottleneck that is about to disappear. Two questions organize what remains: how an automated experimenter can be a trustworthy one, and what becomes askable once it is — trust and reach. Neither is a matter of building the system; the engineering belongs to the prototype, and these questions outlast it. They are also coupled: reach without trust yields evidence nobody can believe, and trust without reach is the careful, occasional benchmarking we already practise.

5.1 A Methodology for Non-Human Experimenters

How does a tireless experimenter remain an honest one — and how much of an inquiry must be reproducible: the experiment, or the path that led to it?

5.2 What Becomes Possible

What can be asked once designing, running, and interpreting an experiment is cheap?

6 Impact and Adoption

Who conducts benchmark studies once an agent can? The answer shapes what the contract must express. A first group operates its own clusters. A thesis comparing three storage engines today spends its first weeks on benchmarking mechanics; with an agent-experimenter, those weeks go into the questions. A vendor's release gate stops reporting red or green and starts reporting that Q9 regressed — and why. Platform and database-as-a-service teams, who benchmark rarely because every campaign is expensive, can ask "does our workload survive the move from local NVMe to Ceph?" as a matter of routine. Consultants can answer with measurements on the client's own workload instead of vendor claims.

A second group is easily overlooked: machines. Once experiments are cheap and contractual, a tuning agent can request a controlled evaluation before committing a knob change, a diagnosis agent can verify its hypothesis, and a capacity planner can test its projection — benchmarking as a tool call. Section 2 argued that tuning systems rebuild bespoke harnesses; under this vision they become *clients*. A service whose primary consumers are other services is what AI-readiness means taken seriously.

A third group is institutional. Artifact evaluation committees can treat replication as an executable check rather than a badge: a reviewer's agent re-runs a study from its archived specification and reports whether the claims hold elsewhere. And students can design experiments and critique validity instead of fighting infrastructure — arguably the better pedagogy for experimental methodology.

Two adoption modes cut across these groups and stress different parts of the contract. *Democratization* brings users with performance questions but no benchmarking craft; they depend on the validity guardrails, because they cannot recognize an invalid experiment themselves. *Amplification* lets experts run an order of magnitude more studies; they depend on the archive and on diffable specifications to keep a growing body of work coherent. One friction deserves naming: several commercial licenses restrict publishing benchmark results, and benchmarking at near-zero marginal cost collides with that tradition — a tension the community will have to negotiate, not one the contract can resolve.

7 Conclusion

Benchmarking automated its execution years ago; this paper argues for automating its experimenter. The enabling step is deliberately small: a two-sided, enforced contract — an intent-bearing catalog of workloads and systems in, navigable and validity-first results out — that turns benchmark studies from human-triggered campaigns into a standing capability, for people and for other machines. Binding the experimenter to a contract also confines its nondeterminism to design time: the study it produces is as replayable as any human-authored one, and more strictly specified. Our thesis, made testable by the prototype and its planned ablations, is that AI-readiness is a property of interfaces, not of intelligence. If it holds, we expect the benchmark papers of the early 2030s to ship with an agent-runnable specification — and the experimenter itself to become a citable, replayable artifact.

Acknowledgments

References

[1] Dana Van Aken, Andrew Pavlo, Geoffrey J. Gordon, and Bohan Zhang. 2017. Automatic Database Management System Tuning Through Large-scale Machine Learning. In *Proceedings of the 2017 International Conference on Management of Data (SIGMOD)*. ACM, 1009–1024. doi:10.1145/3035918.3064029

[2] Mubashara Akhtar, Omar Benjelloun, Costanza Conforti, Pieter Gijsbers, Joan Giner-Miguelez, Nitisha Jain, Michael Kuchnik, Quentin Lhoest, Pierre Marcenac, Manil Maskey, et al. 2024. Croissant: A Metadata Format for ML-Ready Datasets. In *Proceedings of the Eighth Workshop on Data Management for End-to-End Machine Learning (DEEM@SIGMOD)*. ACM, 1–6. doi:10.1145/3650203.3663326

[3] Peter Boncz. 2022. Technical Perspective: The 'Art' of Automatic Benchmark Extraction. *Commun. ACM* 65, 12 (2022), 104. doi:10.1145/3567465

[4] Peter Boncz, Thomas Neumann, and Orri Erling. 2014. TPC-H Analyzed: Hidden Messages and Lessons Learned from an Influential Benchmark. In *Performance Evaluation and Benchmarking (TPCTC 2013) (Lecture Notes in Computer Science, Vol. 8391)*. Springer, 61–76. doi:10.1007/978-3-319-04936-6_5

[5] Yinfang Chen, Manish Shetty, Gagan Somashekar, Minghua Ma, Yogesh Simmhan, Jonathan Mace, Chetan Bansal, Rujia Wang, and Saravan Rajmohan. 2025. AIOpsLab: A Holistic Framework to Evaluate AI Agents for Enabling Autonomous Clouds. In *Proceedings of Machine Learning and Systems (MLSys)*, Vol. 7.

[6] Sunyanan Choochotkaew, Tatsuhiro Chiba, Scott Trent, Takeshi Yoshimura, and Marcelo Amaral. 2022. AutoDECK: Automated Declarative Performance Evaluation and Tuning Framework on Kubernetes. In *2022 IEEE 15th International Conference on Cloud Computing (CLOUD)*. IEEE, 309–314. doi:10.1109/CLOUD55607.2022.00052

[7] Shaleen Deep, Anja Gruenheid, Kruthi Nagaraj, Hiro Naito, Jeff Naughton, and Stratis Viglas. 2022. DIAMetrics: Benchmarking Query Engines at Scale. *Commun. ACM* 65, 12 (2022), 105–112. doi:10.1145/3565633

[8] Djellel Eddine Difallah, Andrew Pavlo, Carlo Curino, and Philippe Cudré-Mauroux. 2013. OLTP-Bench: An Extensible Testbed for Benchmarking Relational Databases. *Proceedings of the VLDB Endowment* 7, 4 (2013), 277–288. doi:10.14778/2732240.2732246

[9] Patrick K. Erdelt. 2021. A Framework for Supporting Repetition and Evaluation in the Process of Cloud-Based DBMS Performance Benchmarking. In *Performance Evaluation and Benchmarking (TPCTC 2020) (Lecture Notes in Computer Science, Vol. 12752)*. Springer, 75–92. doi:10.1007/978-3-030-84924-5_6

[10] Patrick K. Erdelt. 2022. Orchestrating DBMS Benchmarking in the Cloud with Kubernetes. In *Performance Evaluation and Benchmarking (TPCTC 2021) (Lecture Notes in Computer Science, Vol. 13169)*. Springer, 81–97. doi:10.1007/978-3-030-94437-7_6

[11] Patrick K. Erdelt. 2024. A Cloud-Native Adoption of Classical DBMS Performance Benchmarks and Tools. In *Performance Evaluation and Benchmarking (TPCTC 2023) (Lecture Notes in Computer Science, Vol. 14247)*. Springer, 124–142. doi:10.1007/978-3-031-68031-1_9

[12] Patrick K. Erdelt and Jascha Jestel. 2022. DBMS-Benchmarker: Benchmark and Evaluate DBMS in Python. *Journal of Open Source Software* 7, 79 (2022), 4628. doi:10.21105/joss.04628

[13] Michael Fruth, Stefanie Scherzinger, Wolfgang Mauerer, and Ralf Ramsauer. 2022. Tell-Tale Tail Latencies: Pitfalls and Perils in Database Benchmarking. In *Performance Evaluation and Benchmarking (TPCTC 2021) (Lecture Notes in Computer Science, Vol. 13169)*. Springer, 119–134. doi:10.1007/978-3-030-94437-7_8

[14] Victor Giannakouris and Immanuel Trummer. 2025. λ-Tune: Harnessing Large Language Models for Automated Database System Tuning. *Proceedings of the ACM on Management of Data (SIGMOD)* 3, 1 (2025). doi:10.1145/3709652

[15] Sören Henning and Wilhelm Hasselbring. 2021. Theodolite: Scalability Benchmarking of Distributed Stream Processing Engines in Microservice Architectures. *Big Data Research* 25 (2021), 100209. doi:10.1016/j.bdr.2021.100209

[16] Xinmei Huang, Haoyang Li, Jing Zhang, Xinxin Zhao, Zhiming Yao, Yiyan Li, Tieying Zhang, Jianjun Chen, Hong Chen, and Cuiping Li. 2025. E2ETune: End-to-End Knob Tuning via Fine-Tuned Generative Language Model. *Proceedings of the VLDB Endowment* 18, 13 (2025), 5540–5554. doi:10.14778/3773731.3773732

[17] Saurabh Jha, Rohan Arora, Yuji Watanabe, Takumi Yanagawa, Yinfang Chen, Jackson Clark, Bhavya Bhavya, Mudit Verma, Harshit Kumar, et al. 2025. ITBench: Evaluating AI Agents across Diverse Real-World IT Automation Tasks. arXiv preprint arXiv:2502.05352.

[18] Michiel Van Kenhove, Merlijn Sebrechts, Filip De Turck, and Bruno Volckaert. 2023. Cloud-Native-Bench: an Extensible Benchmarking Framework to Streamline Cloud Performance Tests. In *2023 IEEE 12th International Conference on Cloud Networking (CloudNet)*. IEEE, 86–93. doi:10.1109/CloudNet59005.2023.10490079

[19] Charalampos Kostopoulos, George Mouchakis, Antonis Troumpoukis, Nefeli Prokopaki-Kostopoulou, Angelos Charalambidis, and Stasinos Konstantopoulos. 2021. KOBE: Cloud-Native Open Benchmarking Engine for Federated Query Processors. In *The Semantic Web (ESWC 2021) (Lecture Notes in Computer Science, Vol. 12731)*. Springer, 664–679. doi:10.1007/978-3-030-77385-4_40

[20] Eugenie Lai et al. 2025. KramaBench: A Benchmark for AI Systems on Data-to-Insight Pipelines over Data Lakes. arXiv preprint arXiv:2506.06541.

[21] Jiale Lao, Yibo Wang, Yufei Li, Jianping Wang, Yunjia Zhang, Zhiyuan Cheng, Wanghu Chen, Mingjie Tang, and Jianguo Wang. 2024. GPTuner: A Manual-Reading Database Tuning System via GPT-Guided Bayesian Optimization. *Proceedings of the VLDB Endowment* 17, 8 (2024), 1939–1952. doi:10.14778/3659437.3659449

[22] Yiyan Li, Haoyang Li, Jing Zhang, Renata Borovica-Gajic, Shuai Wang, Tieying Zhang, Jianjun Chen, Rui Shi, Cuiping Li, and Hong Chen. 2025. AgentTune: An Agent-Based Large Language Model Framework for Database Knob Tuning. *Proceedings of the ACM on Management of Data (SIGMOD)* 3, 6 (2025). doi:10.1145/3769758 Article 293.

[23] Wan Shen Lim, Matthew Butrovich, William Zhang, Andrew Crotty, Lin Ma, Peijing Xu, Johannes Gehrke, and Andrew Pavlo. 2023. Database Gyms. In *Conference on Innovative Data Systems Research (CIDR)*.

[24] Qi Lin, Zhenyu Zhang, Viraj Thakkar, Zhenjie Sun, Mai Zheng, and Zhichao Cao. 2025. StorageXTuner: An LLM Agent-Driven Automatic Tuning Framework for Heterogeneous Storage Systems. arXiv preprint arXiv:2510.25017.

[25] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. 2024. The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery. arXiv preprint arXiv:2408.06292.

[26] MLCommons Science Working Group. 2025. An MLCommons Scientific Benchmarks Ontology. arXiv preprint arXiv:2511.05614.

[27] Alessandro Vittorio Papadopoulos, Laurens Versluis, André Bauer, Nikolas Herbst, Jóakim von Kistowski, Ahmed Ali-Eldin, Cristina L. Abad, José Nelson Amaral, Petr Tůma, and Alexandru Iosup. 2021. Methodological Principles for Reproducible Performance Evaluation in Cloud Computing. *IEEE Transactions on Software Engineering* 47, 8 (2021), 1528–1543. doi:10.1109/TSE.2019.2927908

[28] Mark Raasveldt, Pedro Holanda, Tim Gubner, and Hannes Mühleisen. 2018. Fair Benchmarking Considered Difficult: Common Pitfalls in Database Performance Testing. In *Proceedings of the Workshop on Testing Database Systems (DBTest)*. ACM, 2:1–2:6. doi:10.1145/3209950.3209955

[29] Samuel Schmidgall, Yusheng Su, Ze Wang, Ximeng Sun, Jialian Wu, Xiaodong Yu, Jiang Liu, Zicheng Liu, and Emad Barsoum. 2025. Agent Laboratory: Using LLM Agents as Research Assistants. arXiv preprint arXiv:2501.04227.

[30] Daniel Seybold and Jörg Domaschka. 2021. Automated Benchmarking of DBMS with benchANT. In *Proceedings of the Conference on Innovative Data Systems and Applications (poster) (CEUR Workshop Proceedings, Vol. 3043)*.

[31] Daniel Seybold, Moritz Keppler, Daniel Gründler, and Jörg Domaschka. 2019. Mowgli: Finding Your Way in the DBMS Jungle. In *Proceedings of the 2019 ACM/SPEC International Conference on Performance Engineering (ICPE)*. ACM, 321–332. doi:10.1145/3297663.3310303

[32] Immanuel Trummer. 2022. DB-BERT: A Database Tuning Tool that "Reads the Manual". In *Proceedings of the 2022 International Conference on Management of Data (SIGMOD)*. ACM, 190–203. doi:10.1145/3514221.3517843

[33] Johannes Wehrstein, Timo Eckmann, Matthias Jasny, and Carsten Binnig. 2026. Bespoke OLAP: Synthesizing Workload-Specific One-size-fits-one Database Engines. arXiv preprint arXiv:2603.02001.

[34] Johannes Wehrstein, Anton Winter, Timo Eckmann, and Carsten Binnig. 2026. Bespoke-Card: Why Tune When You Can Generate? Synthesizing Workload-Specific Cardinality Estimators. arXiv preprint arXiv:2606.09361.

[35] Mark D. Wilkinson, Michel Dumontier, IJsbrand Jan Aalbersberg, et al. 2016. The FAIR Guiding Principles for Scientific Data Management and Stewardship. *Scientific Data* 3 (2016), 160018. doi:10.1038/sdata.2016.18

[36] Christopher Zanoli, Andrea Giovannini, Tengjun Jin, Ana Klimovic, and Yotam Perlitz. 2026. ELT-Bench-Verified: Benchmark Quality Issues Underestimate AI Agent Capabilities. arXiv preprint arXiv:2603.29399.

[37] Xinyi Zhang, Tiantian Chen, Zhentao Han, Zhaoyan Hong, Wei Lu, Sheng Wang, Mo Sha, Anni Wang, Shuang Liu, Yakun Zhang, Feifei Li, and Xiaoyong Du. 2026. Why Database Manuals are Not Enough: Efficient and Reliable Configuration Tuning for DBMSs via Code-Driven LLM Agents. *Proceedings of the VLDB Endowment* 19, 6 (2026), 1358–1371. doi:10.14778/3797919.3797940

[38] Xuanhe Zhou, Guoliang Li, Zhaoyan Sun, Zhiyuan Liu, Weize Chen, Jianming Wu, Jiesi Liu, Ruohang Feng, and Guoyang Zeng. 2024. D-Bot: Database Diagnosis System using Large Language Models. *Proceedings of the VLDB Endowment* 17, 10 (2024), 2514–2527. doi:10.14778/3675034.3675043

[39] Linwu Zhu, Liqiang Gao, Yan Chen, Dan Zhu, and Jian Huang. 2026. LAP: An Agent-to-Instrument Protocol for Autonomous Science. arXiv preprint arXiv:2606.03755.